

**ConstructoCompare Pro**

**Planes de Pruebas Funcionales**

Pruebas de Criterios de Aceptación

Versión: 5.0

Fecha: 31/05/2026

**HOJA DE CONTROL**

|                        |                                                                  |
|------------------------|------------------------------------------------------------------|
| <b>Cliente</b>         | Proyecto ConstructoCompare Pro                                   |
| <b>Proyecto</b>        | ConstructoCompare Pro                                            |
| <b>Entregable</b>      | Plan de Pruebas Funcionales – Pruebas de Criterios de Aceptación |
| <b>Autor</b>           | Genara Alarcón (Q/A)                                             |
| <b>Versión/Edición</b> | 5.0 (Sprints 1–5 completos)                                      |
| <b>Fecha</b>           | 31/05/2026                                                       |

**REGISTRO DE CAMBIOS**

| <b>Versión</b> | <b>Causa del Cambio</b>                | <b>Responsable del Cambio</b> | <b>Fecha del Cambio</b> |
|----------------|----------------------------------------|-------------------------------|-------------------------|
| 001            | Versión inicial – Sprints 1 a 4        | Genara Alarcón                | 30/03/2026              |
| 004            | Adición Sprint 4 y estructura Sprint 5 | Genara Alarcón                | 31/05/2026              |

**CONTROL DE DISTRIBUCIÓN**

| <b>Nombre y Apellidos</b> | <b>Rol</b>                       |
|---------------------------|----------------------------------|
| Javiera Ramírez           | Product Owner / Data Engineer    |
| Samuel Bueno              | Scrum Master / Backend Developer |
| Israel Gonzalez           | Frontend Developer / UX Designer |
| Genara Alarcón            | QA Engineer / DevOps             |

# ÍNDICE

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1 INTRODUCCIÓN</b>                                            | <b>4</b>  |
| 1.1 Objeto                                                       | 4         |
| 1.2 Alcance                                                      | 4         |
| <b>2 TRAZABILIDAD DE CASOS DE PRUEBAS – HISTORIAS DE USUARIO</b> | <b>5</b>  |
| <b>3 DEFINICIÓN DE LOS CASOS DE PRUEBAS</b>                      | <b>6</b>  |
| 3.1 Sprint 1 – Infraestructura Base                              | 6         |
| 3.1.1 Backend                                                    | 6         |
| 3.1.2 Frontend                                                   | 7         |
| 3.1.3 Scraping                                                   | 7         |
| 3.2 Sprint 2 – Motor Asíncrono y Visualización                   | 8         |
| 3.2.1 Backend                                                    | 8         |
| 3.2.2 Frontend                                                   | 8         |
| 3.2.3 Scraping                                                   | 9         |
| 3.3 Sprint 3 – Motor de Comparación                              | 10        |
| 3.3.1 Backend                                                    | 10        |
| 3.3.2 Frontend                                                   | 10        |
| 3.3.3 Scraping                                                   | 11        |
| 3.4 Sprint 4 – Dashboard de Cotizaciones                         | 12        |
| 3.4.1 Backend                                                    | 12        |
| 3.4.2 Frontend                                                   | 13        |
| 3.4.3 Scraping                                                   | 13        |
| 3.5 Sprint 5 – Perfil de Usuario, Export PDF y Dashboard Final   | 14        |
| 3.5.1 Backend                                                    | 14        |
| 3.5.2 Frontend – Pendiente                                       | 16        |
| 3.5.3 Scraping – Pendiente                                       | 16        |
| <b>4 ESTRATEGIA DE EJECUCIÓN DE PRUEBAS</b>                      | <b>17</b> |
| 4.1 Tipos de Prueba Utilizados                                   | 17        |
| 4.2 Herramientas                                                 | 17        |
| 4.3 Criterios de Aceptación Generales                            | 17        |
| <b>5 ANEXOS</b>                                                  | <b>18</b> |
| <b>6 GLOSARIO</b>                                                | <b>19</b> |

# 1 INTRODUCCIÓN

## 1.1 Objeto

Este documento recoge los casos de prueba que verifican que el sistema ConstructoCompare Pro satisface los Criterios de Aceptación de las Historias de Usuario definidas por sprint. Contiene la definición de los casos de prueba, la trazabilidad hacia los requisitos funcionales y la estrategia de ejecución seguida por el equipo de QA.

## 1.2 Alcance

El alcance de este plan cubre cinco sprints completados (S1–S5). En cada sprint se validan tres capas críticas:

- Backend (FastAPI): verificación de esquemas, persistencia, seguridad JWT y rendimiento de endpoints.
- Frontend (Next.js): flujos de interacción, UI, renderizado de datos y cargas asíncronas.
- Web Scraping (Playwright): calidad en la ingesta, sanitización de strings y consistencia del histórico de precios.

Responsable de QA: Genara Alarcón. Período: 30/03/2026 – 31/05/2026.

## 2 TRAZABILIDAD DE CASOS DE PRUEBAS – HISTORIAS DE USUARIO

La siguiente tabla relaciona las Historias de Usuario (HU) con los casos de prueba ejecutados en cada sprint. Los identificadores de HU siguen la nomenclatura del proyecto (HU-Jx, HU-Sx, HU-Gx, HU-Ix).

| Sprint | ID HU    | Capa     | Historia de Usuario                                  | Casos de Prueba      |
|--------|----------|----------|------------------------------------------------------|----------------------|
| S1     | HU-J1    | Backend  | Infraestructura de Datos y Persistencia de Usuarios  | TC-S1-BE-01/02       |
| S1     | HU-I1    | Frontend | Desarrollo de Interfaz de Usuario e Interacción Base | TC-S1-01/02/03       |
| S1     | HU-S1/G1 | Scraping | Motor de Búsqueda y Aseguramiento de Calidad         | TC-S1-SC-01/02       |
| S2     | HU-S2/J2 | Scraping | Motor Asíncrono y Sanitización de Precios            | TC-S2-SC-01/02       |
| S2     | HU-G2    | Scraping | QA de Calidad de Ingesta y Logs                      | TC-S2-SC-03          |
| S2     | HU-I2    | Frontend | Visualización de Datos Reales y Lazy Loading         | TC-S2-01/02/03/04    |
| S3     | HU-J3    | Backend  | Arquitectura de Unificación (ID Maestro)             | TC-S3-BE-01/02       |
| S3     | HU-S3    | Scraping | Pipeline Medallion y Matching Semántico              | TC-S3-SC-01/02       |
| S3     | HU-I3    | Frontend | Vista Side-by-Side y Resaltado Pro                   | TC-S3-01/02/03       |
| S4     | HU-J4    | Backend  | Modelo de Persistencia y Soft Delete                 | TC-S4-BE-01/02/03    |
| S4     | HU-S4    | Backend  | CRUD API de Gestión de Cotizaciones                  | TC-S4-BE-04          |
| S4     | HU-I4    | Frontend | Dashboard Interactivo y Edición de Cantidades        | TC-S4-01/02/03/04/05 |
| S5     | HU-J5    | Backend  | Perfil de Usuario y Despliegue Cloud                 | TC-S5-QA-01          |
| S5     | HU-I5    | Frontend | Dashboard Final y Gráficos Históricos                | TC-S5-01/02/03       |

### 3 DEFINICIÓN DE LOS CASOS DE PRUEBAS

A continuación se detallan los casos de prueba ejecutados por sprint, organizados por capa (Backend, Frontend, Scraping), con sus resultados y conclusiones.

#### 3.1 Sprint 1 – Infraestructura Base

Período de ejecución: 30/03/2026

| ID    | Épica         | Sprint | Historia de Usuario                                    |
|-------|---------------|--------|--------------------------------------------------------|
| HU-J1 | Panel Usuario | S1     | Infraestructura de Datos y Persistencia de Usuarios    |
| HU-S1 | Scraping      | S1     | Implementación del Motor de Búsqueda y Extracción Base |
| HU-I1 | Panel Usuario | S1     | Desarrollo de Interfaz de Usuario e Interacción Base   |
| HU-G1 | Scraping      | S1     | Aseguramiento de Calidad, Integridad y Seguridad       |

##### 3.1.1 Backend

###### Servicio de Autenticación – Prueba de Humo (TC-S1-BE-01)

Objetivo: verificar que los endpoints del backend respondan con los códigos HTTP correctos y que el flujo de registro bloquee anomalías antes de persistir información.

Casos ejecutados:

- Disponibilidad del servidor (Smoke Test API): GET a la raíz de la API para certificar que FastAPI esté encendido. Resultado: PASSED.
- Registro exitoso de usuario nuevo: payload con datos válidos y correo único (uuid). La API responde con HTTP 200/201. Resultado: PASSED.
- Rechazo de correos duplicados (Mocking): uso de MagicMock y AsyncMock para interceptar la consulta y simular correo ya existente. Resultado: PASSED.

Conclusión: los endpoints de registro cumplen las políticas de negocio. Las dependencias simuladas (Dependency Overrides y AsyncMocks) aíslan la lógica de la API de la capa de persistencia, logrando tiempos de ejecución menores a 0,85 s con respuestas HTTP estandarizadas.

###### Prueba de Estrés y Concurrencia – Login (TC-S1-BE-02)

Objetivo: evaluar la resiliencia del endpoint de login ante picos masivos de tráfico concurrente.

- Simulación de 100 usuarios simultáneos atacando /api/v1/users/login mediante hilos. Resultado: 100 respuestas 401 (credencial de prueba no registrada); 0 errores 500; tiempo total < 10 s. PASSED.

Análisis de carga (Locust): carga incremental de 50 a 200 usuarios concurrentes:

- RPS Máximo: 1,97 req/s (límite de hardware por coste de Bcrypt).
- Latencia p95: 45.000 ms (coste computacional de encriptación).
- Fail Rate: 33 % a 200 usuarios (saturación del pool de conexiones).

Conclusión: arquitectura FastAPI resiliente a picos reales de tráfico. Tasa de fallos críticos del 0 % en servidor. La API está preparada para producción con picos controlados.

### 3.1.2 Frontend

| ID       | Tipo   | Nombre                           | Descripción                                                                                                             | HUs                                |
|----------|--------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| TC-S1-01 | Humo   | Flujo completo login→búsqueda→UF | Login con credenciales demo, inyección de sesión, navegación a /app, verificación UF/monetarios, búsqueda de "cemento". | HU8-AC2, HU8-AC3, HU1-AC2, HU6-AC3 |
| TC-S1-02 | Estrés | 5 intentos login fallido         | 5 logins consecutivos con credenciales incorrectas; verifica rechazo en < 10 s.                                         | HU8-AC2                            |
| TC-S1-03 | Humo   | Validación formulario registro   | 3 escenarios inválidos: email incorrecto, contraseña corta y campos vacíos.                                             | HU8-AC1                            |

Conclusión: la ejecución automatizada con Playwright demuestra cumplimiento normativo de los criterios de aceptación del Sprint 1 en frontend. Sesión inmediata, renderizado adaptativo de tarjetas y reglas de validación en formularios. Todos los casos: PASSED.

### 3.1.3 Scraping

Módulo: Scraper Sodimac (Sodimac.py). Python + Playwright + Asyncio. Funciones clave: búsqueda inteligente por sitemap.xml, navegación automatizada, procesamiento en paralelo (workers), captura estructurada (Nombre, Marca, SKU, URL, precios Normal/Internet/O oferta/CMR) y sistema de reintentos.

#### Prueba de Humo (TC-S1-SC-01)

- Criterio 1 & 2: conexión a Sodimac y recuperación de productos por palabra clave. PASSED.
- Criterio 3: selectores CSS funcionales con datos esenciales (nombre + precio). PASSED.
- Criterio 4: limpieza de memoria entre búsquedas y resultados actualizados. PASSED.

Conclusión: arquitectura asíncrona basada en Playwright completamente operativa. Respeta robots.txt y el sistema está listo para extracción a gran escala.

#### Prueba de Estrés y Eficiencia (TC-S1-SC-02)

Configuración: 10 temáticas, máximo 500 productos, 40 categorías, 5 workers simultáneos.

Resultado: velocidad proyectada ~80.357 productos/hora (supera la meta de ~70.000). Pico de CPU controlado. Incremento de RAM < 1 GB.

Conclusión: el software supera la meta planteada. Viabilidad de despliegue continuo garantizada.

## 3.2 Sprint 2 – Motor Asíncrono y Visualización

| ID      | Épica         | Sprint | Historia de Usuario                          |
|---------|---------------|--------|----------------------------------------------|
| HU-S2.1 | Scraping      | S2     | Motor Asíncrono y Extracción Sodimac         |
| HU-J2.1 | Scraping      | S2     | Sanitización y Normalización de Precios      |
| HU-G2.1 | Scraping      | S2     | QA de Calidad de Ingesta y Logs              |
| HU-I2.1 | Panel Usuario | S2     | Visualización de Datos Reales y Lazy Loading |

### 3.2.1 Backend

Durante el Sprint 2 no se ejecutaron suites de pruebas automatizadas específicas sobre el código del backend. Esta decisión responde a una priorización estratégica: el núcleo de desarrollo se concentró en el Motor de Scraping y la Capa de Calidad de Datos (Silver). El esfuerzo de QA se enfocó en: (a) validación de resiliencia multitienda (Sodimac, Easy, Imperial), (b) pruebas de rendimiento con 15 workers asíncronos, y (c) auditoría de la capa Silver.

Próximos pasos: las pruebas de endpoints serán reactivadas en el siguiente ciclo una vez consolidada la madurez del motor de extracción.

### 3.2.2 Frontend

Control de Calidad e Integración de Datos

| ID       | Tipo   | Nombre                                | Descripción                                                                                                    | HUs                  |
|----------|--------|---------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------|
| TC-S2-01 | Humo   | Carga datos reales sin interrupciones | Búsqueda de "fierro", tiempo < 5 s, verificación de ausencia de errores en pantalla, scroll para lazy loading. | HU-I2.1-AC1, AC2     |
| TC-S2-02 | Estrés | 10 búsquedas consecutivas             | 10 términos distintos; tiempo máximo < 5 s; degradación entre primera y segunda mitad < 2 s.                   | HU-I2.1-AC1, AC4     |
| TC-S2-03 | Humo   | Datos en tarjetas                     | Primeras 3 tarjetas con nombre, precio (\$) y tienda (Sodimac, Easy o Imperial).                               | HU-I2.1-AC3, HU1-AC2 |
| TC-S2-04 | Estrés | 3 usuarios simultáneos                | 3 hilos en paralelo; HTTP 200 en todos; tiempo máximo < 8 s.                                                   | HU-I2.1-AC1, AC4     |

Hallazgo TC-S2-03: AssertionError por desfase de sincronización (timing) entre el motor de pruebas y el ciclo de renderizado del frontend. El contenedor existe en el DOM pero `.inner_text()` se ejecuta antes de que la lógica asíncrona inyecte los datos.

Mitigaciones sugeridas: (a) esperas explícitas por sub-elementos específicos (precio o título), (b) pausa de consistencia tras detectar el contenedor.

Conclusión: alto cumplimiento de criterios de aceptación. El hallazgo de timing en TC-S2-03 será resuelto con las mitigaciones planteadas. TC-S2-01, TC-S2-02 y TC-S2-04: PASSED.

### **3.2.3 Scraping**

#### **Prueba de Humo y Resiliencia Multitienda (TC-S2-SC-01)**

Mecanismo de reintento sobre palabras clave estratégicas validando nombre, SKU y al menos una variante de precio. Orquestación de SodimacScraper, EasyScraper e ImperialScraper con estado lógico unificado.

Resultado: PASSED. Imperial migró autónomamente a una categoría alternativa ante categoría vacía, demostrando alta resiliencia.

#### **Prueba de Estrés – Métricas de Hardware (TC-S2-SC-02)**

15 hilos concurrentes por tienda. Monitoreo de CPU y RAM cada 1 segundo (psutil).

Resultados: 359,52 productos/minuto promedio → proyección > 21.500 productos/hora. CPU promedio: 42,5 %. RAM pico: 3,42 GB. Sin cuellos de botella.

#### **Control de Calidad de Datos – Capa Silver (TC-S2-SC-03)**

Validaciones: (1) formato numérico de precios (int o float, sin ceros ni negativos), (2) sanitización de strings sin espacios dobles ni blancos, (3) cobertura multitienda con registros > 0 en Sodimac, Easy e Imperial.

Resultado: 3 passed (100 %). Base de datos estandarizada y lista para explotación segura en backend y frontend.



### 3.3 Sprint 3 – Motor de Comparación

| ID      | Épica             | Sprint | Historia de Usuario                      |
|---------|-------------------|--------|------------------------------------------|
| HU-J3.1 | Motor Comparación | S3     | Arquitectura de Unificación (ID Maestro) |
| HU-S3.1 | Motor Comparación | S3     | API de Comparación y Lógica is_cheapest  |
| HU-G3.1 | Motor Comparación | S3     | QA de Matching y Tuning SQL              |
| HU-I3.1 | Motor Comparación | S3     | Vista Side-by-Side y Resaltado Pro       |

#### 3.3.1 Backend

##### Control de Calidad e Integración (TC-S3-BE-01)

Foco en consistencia transaccional, mapeo correcto de tipos e integridad relacional del motor de persistencia al procesar información unificada por el scraper.

##### Prueba de Estrés Concurrente – Motor de Búsqueda (TC-S3-BE-01)

50 peticiones simultáneas mediante `asyncio.gather` + HTTPX sobre `/api/v1/inventory/search`.

Resultado: 0 errores 500. Pool de conexiones PostgreSQL operativo bajo carga. PASSED.

Conclusión: alta madurez arquitectónica asíncrona. FastAPI/HTTPX y PostgreSQL operan coordinadamente ante picos de tráfico.

##### Validación de Matching y Agrupación (TC-S3-BE-02)

Certifica que el endpoint de consulta agrupe productos bajo un maestro común y no duplique ofertas del mismo retail (assert de unicidad de conjuntos).

Resultado: PASSED. Motor de matching unifica inventarios dispersos en registros limpios y libres de redundancias.

Estado final: 100 % PASSED (2 passed).

#### 3.3.2 Frontend

##### Control de Calidad e Inteligencia de Precios

| ID       | Tipo   | Nombre                                 | Descripción                                                                                                     | HUs              |
|----------|--------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------|
| TC-S3-01 | Humo   | Badge Mejor Precio automático          | Busca badge "Mejor Precio" por múltiples selectores CSS; fallback a vista comparativa multi-tienda con precios. | HU-I3.1, HU3-AC1 |
| TC-S3-02 | Humo   | Switch CLP/UF actualiza precios        | Toggle UF: activa/desactiva y verifica cambios en la representación de precios.                                 | HU-I3.1, HU6-AC2 |
| TC-S3-03 | Estrés | Badge estable bajo 5 cambios de filtro | 5 filtros de búsqueda sucesivos; sin errores 500, JS crashes; tiempo < 6 s.                                     | HU-I3.1, HU3-AC2 |

Conclusión: 100 % PASSED. El sistema procesa lógicas comparativas complejas, despliega badges analíticos de costos y responde con fluidez a la conmutación CLP/UF. Estabilidad óptima bajo filtrado continuo.

### **3.3.3 Scraping**

#### **Prueba de Humo – Arquitectura Medallion (TC-S3-SC-01)**

Validación del pipeline Bronze → Silver → Gold. El Motor Gold aplica `threshold_confident=50` para identificar variantes por tienda de un único producto maestro canónico. URLs físicas mapeadas y auditables.

Resultado: PASSED. Pipeline Medallion cumple metas del Sprint 3.

#### **Prueba de Estrés E2E del Pipeline (TC-S3-SC-02)**

Fase Bronze: ingesta masiva concurrente con `monitor_resources` (telemetría cada 1 s).

Fase Silver: remoción de ruido. Fase Gold: matching y agrupación por Productos Maestros.

Métricas: tiempo total pipeline 56,81 s. Velocidad: 605,19 productos/min → proyección > 36.311 registros/hora. CPU promedio 49,0 % (pico 99,2 %). RAM máxima 19,00 GB. Tiempo Silver: 0,11 s. Tiempo Gold: 0,06 s.

Conclusión: sistema preparado para cargas a escala comercial. Datos fluyen limpios con conformidad técnica para entornos productivos de alta concurrencia.

### 3.4 Sprint 4 – Dashboard de Cotizaciones

| ID      | Épica         | Sprint | Historia de Usuario                           |
|---------|---------------|--------|-----------------------------------------------|
| HU-J4.1 | Panel Usuario | S4     | Modelo de Persistencia y Soft Delete          |
| HU-S4.1 | Panel Usuario | S4     | CRUD API de Gestión de Cotizaciones           |
| HU-G4.1 | Panel Usuario | S4     | QA Integridad Financiera y Pruebas de Carga   |
| HU-I4.1 | Panel Usuario | S4     | Dashboard Interactivo y Edición de Cantidades |

#### 3.4.1 Backend

##### Prueba de Humo – Ciclo de Vida de Cotizaciones (TC-S4-BE-01)

Secuencia de integración: inyección de JSON complejo con IDs de productos y cantidades → validación de creación (HTTP 201/200) → petición de lectura inmediata para comprobar indexación en el historial del perfil.

Resultado: PASSED. Sin pérdidas ni truncados. Endpoint transaccionalmente seguro.

##### Resiliencia Concurrente en Escritura (TC-S4-BE-02)

50 peticiones POST simultáneas sobre /api/v1/quotes. Aserciones cruzadas: respuestas exitosas = peticiones emitidas.

Resultado: 50 escrituras exitosas. 0 errores 500. PASSED.

##### Validación CRUD Completo Inter-Servicios (TC-S4-BE-03)

Flujo E2E: registro/login → token JWT → consulta inventario → crear cotización → listar → leer por ID → actualizar (nombre + cantidad) → eliminar → verificar que ID no existe.

Resultado: PASSED. Microservicio de cotizaciones interactúa correctamente con el ecosistema backend.

##### Auditoría de Carga (Locust)

50 a 200 usuarios concurrentes ejecutando flujos de registro, autenticación, lectura y creación de presupuestos. Resultados de escalabilidad documentados en el informe de carga adjunto.

Conclusión backend: suite certifica correcta implementación del Sprint 4. API robusta para flujos encadenados de lectura/escritura asíncrona con arquitectura ASGI. 0 excepciones de persistencia.

### 3.4.2 Frontend

#### Pruebas E2E – Dashboard de Cotizaciones

| ID       | Tipo          | Nombre                                 | Descripción                                                                                                | HUs                          |
|----------|---------------|----------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------|
| TC-S4-01 | Humo          | Panel cotización carga y muestra ítems | Valida acceso al panel lateral desde navbar y carga inicial limpia en < 3 s sin errores visuales.          | HU-I4.1 (AC3, AC4)           |
| TC-S4-02 | Estrés        | Múltiples +/- sin degradación          | 10 clics rápidos alternados en +/- para evaluar reactividad del estado y recálculo de subtotales.          | HU-I4.1 (AC1, AC2, AC3)      |
| TC-S4-03 | Fuego (Spike) | Pico de usuarios simultáneos           | 5 accesos concurrentes con hilos Python para medir estabilidad del hosting Vercel.                         | HU-I4.1 (AC3, AC4)           |
| TC-S4-04 | Regresión     | Consistencia funcional S1–S4           | Login, Registro, Buscador, Precios CLP/UF operando al 100 % tras inyección del ecosistema de cotizaciones. | HU-I4.1 + Regresiones        |
| TC-S4-05 | Volumen       | Panel con muchos ítems                 | 5 materiales distintos inyectados; sumatoria total sin congelamientos.                                     | HU-I4.1 (AC1, AC2, AC3, AC4) |

Conclusión: 100 % PASSED. Promedio de respuesta por acción: 0,33 s. 0 regresiones detectadas. Módulo de cotizaciones maduro, cohesionado con sprints anteriores y listo para producción.

### 3.4.3 Scraping

Durante el Sprint 4 el módulo de scraping no requirió desarrollo ni ejecución de nuevos casos de prueba. Las capacidades de parseo, evasión de bloqueos y sincronización de catálogos (Sodimac, Easy, Imperial) alcanzaron su criterio de parada y madurez técnica en el ciclo anterior (Sprint 3).

### 3.5 Sprint 5 – Perfil de Usuario, Export PDF y Dashboard Final

| ID      | Épica         | Sprint | Historia de Usuario                      |
|---------|---------------|--------|------------------------------------------|
| HU-J5.1 | Panel Usuario | S5     | Perfil de Usuario y Despliegue Cloud     |
| HU-S5.1 | Scraping      | S5     | Export PDF                               |
| HU-G5.1 | Scraping      | S5     | QA Seguridad Final y Reporte de Métricas |
| HU-I5.1 | Panel Usuario | S5     | Dashboard Final y Gráficos Históricos    |

#### 3.5.1 Backend

El aseguramiento de calidad del Sprint 5 abarcó de manera integral dos microservicios: el **User Service** (autenticación e identidad) y el **Cotization Service** (motor de persistencia documental). Las validaciones se dividieron en pruebas funcionales de criterios de aceptación, smoke tests automatizados y estrés de infraestructura.

#### Pruebas Funcionales – User Service

| ID     | Nombre                                        | Resultado             |
|--------|-----------------------------------------------|-----------------------|
| CP-01  | Registro e Integridad de Entidades de Usuario | HTTP 200 OK           |
| CP-02  | Autenticación y Emisión de Credenciales JWT   | HTTP 200 OK           |
| CP-03a | Blindaje Perimetral – Petición Anónima        | HTTP 401 Unauthorized |
| CP-03b | Blindaje Perimetral – Petición Autenticada    | HTTP 200 OK           |
| CP-04  | Ciclo de Vida Completo del Recurso (E2E)      | PASSED                |

**CP-01 – Registro e Integridad:** Valida la regla de negocio que exige la creación unívoca de cuentas, verificando el procesamiento de esquemas JSON y la correcta respuesta del backend. Resultado: HTTP 200 OK.

**CP-02 – Autenticación JWT:** Verifica el flujo de inicio de sesión. El sistema valida el hash Bcrypt almacenado en base de datos y otorga un token de acceso firmado con tiempo de expiración definido. Payload en formato `application/x-www-form-urlencoded`. Resultado: HTTP 200 OK.

**CP-03 – Blindaje Perimetral (OAuth2PasswordBearer):** Contrasta el comportamiento ante peticiones anónimas versus autenticadas sobre el endpoint de modificación de contraseña (PUT). Caso negativo (CP-03a): sin cabecera de autorización → HTTP 401 Unauthorized. Caso positivo (CP-03b): con cabecera `Authorization: Bearer <token_jwt>` → cambio procesado con éxito, nueva clave cifrada con Bcrypt.

**CP-04 – Ciclo de Vida Completo E2E:** Simula el flujo transaccional completo: Registro → Login → Modificación del Perfil → Eliminación del Recurso, validando la ausencia de estados corruptos o registros huérfanos en PostgreSQL.

- Actualización de perfil (PUT /api/v1/users/profile): HTTP 200 OK.
- Eliminación de cuenta (DELETE /api/v1/users/me): HTTP 200 OK. Respuesta: {"message": "Cuenta eliminada exitosamente."}.
- Verificación de baja definitiva: {"detail": "Usuario no encontrado."}. Integridad referencial confirmada.

#### **CP-05 – Smoke Test Automatizado (pytest)**

Script automatizado con pytest que audita la vitalidad del endpoint protegido /api/v1/users/me. El script simula el proceso de login, extrae el token del payload, configura los headers y consulta la ruta privada. Resultado: ejecución limpia. PASSED.

#### **Pruebas Funcionales – Cotization Service**

**CC-01 – Persistencia Dinámica de Cotizaciones:** Valida que al consumir POST /api/v1/cotizaciones, el sistema capture el contexto del usuario autenticado y guarde de forma síncrona los metadatos y listas de materiales. Resultado: HTTP 200 OK / HTTP 201 Created. El objeto de retorno expone un nuevo id\_cotizacion: 104, vinculación al propietario, marcas temporales de auditoría y estado por defecto "Borrador".

**CC-02 – Generación y Streaming de PDF:** Comprueba el pipeline de renderizado y descarga del endpoint GET /api/v1/cotizaciones/{id\_cotizacion}/download. Verificaciones de cabeceras HTTP:

- Status Code: 200 OK.
- Content-Type: application/pdf.
- Content-Disposition: attachment; filename=cotizacion\_104.pdf.

## Pruebas de Carga – Locust

### PE-01 – 50 Usuarios Simultáneos:

| Endpoint                 | # Req. | Fallos | Mediana (ms) | p95 (ms) | p99 (ms) |
|--------------------------|--------|--------|--------------|----------|----------|
| POST /api/v1/users/login | 50     | 3      | 17.000       | 32.000   | 34.000   |
| GET /api/v1/users/me     | 1.216  | 0      | 2.200        | 5.100    | 18.000   |
| Aggregated               | 1.266  | 3      | 2.200        | 14.000   | 28.000   |

Veredicto: **PASS**. Tasa de fallos de red: 0 %. Tiempos de respuesta dentro de umbrales críticos de experiencia de usuario.

### PE-02 – 200 Usuarios Simultáneos:

| Endpoint                 | # Req. | Fallos | Mediana (ms) | p95 (ms) | p99 (ms) |
|--------------------------|--------|--------|--------------|----------|----------|
| POST /api/v1/users/login | 200    | 143    | 34.000       | 36.000   | 41.000   |
| GET /api/v1/users/me     | 1.117  | 5      | 3.000        | 7.800    | 31.000   |
| Aggregated               | 1.317  | 148    | 3.000        | 35.000   | 36.000   |

Tasa de éxito: 89 %. Se registraron errores HTTP 500 por dos cuellos de botella identificados: (1) agotamiento del pool de conexiones de PostgreSQL al alcanzar el límite máximo de conexiones simultáneas, y (2) saturación de CPU por operaciones concurrentes de descifrado y verificación de firmas Bcrypt con factores de trabajo elevados.

### 3.5.2 Frontend

Tabla General de Control de Casos de Prueba

| ID     | Componente              | Flujo                                             | Criterio de Aceptación Vinculado                                 | Veredicto |
|--------|-------------------------|---------------------------------------------------|------------------------------------------------------------------|-----------|
| CPF-01 | Panel Usuario (/perfil) | Renderizado de grilla e historial de cotizaciones | HU-I5.1: Despliegue dinámico de cotizaciones del usuario         | PASS      |
| CPF-02 | Panel Usuario (/perfil) | Interacción con botón "Descargar PDF"             | HU-S5.1: Exportación y streaming correcto de archivos en cliente | PASS      |

#### A. Despliegue de Historial de Cotizaciones en Dashboard (CPF-01)

Verificación del renderizado reactivo del componente de la tabla de cotizaciones. Tras la autenticación exitosa, la interfaz debe capturar el contexto del cliente, consultar los endpoints privados del backend y listar correctamente los registros previos mostrando títulos de proyecto, marcas de tiempo y estados de operación.

- **Datos de Entrada:** Navegación dirigida del usuario autenticado hacia la ruta interna `/perfil` con la sesión activa del token en LocalStorage.
- **Resultados:** Renderizado óptimo de la interfaz del Dashboard sin errores de consola detectados. Despliegue correcto de los bloques de información estructurada correspondientes a los registros almacenados. Resultado: **PASS**.

#### B. Pipeline de Exportación y Descarga Local de PDF (CPF-02)

Validación de la interacción de descarga de documentos. Al hacer clic sobre la acción de exportar, el frontend de Next.js debe consumir el flujo binario (blob) provisto por la API, procesar el buffer en el navegador y forzar la descarga de un archivo PDF estructurado de forma legible.

- **Datos de Entrada:** Clic de usuario sobre el botón "Descargar PDF" asignado a un registro del historial (ej: Proyecto Cotización).
- **Resultados:** Descarga síncrona en cliente de un archivo con extensión `.pdf` completamente estructurado y formateado bajo los parámetros de diseño corporativo. Resultado: **PASS**.



### C. Verificación de Integridad del Documento Generado (cotizacion\_constructocompare.pdf)

Como soporte analítico de calidad sobre el resultado final del Frontend, se auditó el documento binario resultante descargado por la aplicación web, comprobando la fidelidad de los datos dinámicos inyectados desde el DOM y los catálogos en el archivo PDF:

1. Presencia formal de la cabecera "ConstructoCompare PRO", fecha de emisión del reporte validada (31-05-2026) y lectura correcta de la Unidad de Fomento indexada del día (\$40.611).
2. Renderizado limpio de la tabla de materiales extraídos por scraping, estructurado en columnas unificadas (Producto, Tienda, P. Unitario CLP/UF, Cantidad, Subtotal). El documento calcula de manera exacta las conversiones y presenta un cierre de balance consolidado por un valor de **\$191.970 (4.7271 UF)**.

#### 3.5.3 Scraping

Durante el Sprint 5, el módulo de scraping no requirió el desarrollo ni la ejecución de nuevos casos de prueba automatizados. Las capacidades de parseo del DOM, la evasión de bloqueos perimetrales y la sincronización de catálogos (Sodimac, Easy, Imperial) alcanzaron su criterio de parada y madurez técnica en los ciclos anteriores.

Durante esta iteración, el motor de extracción operó de manera transparente como un servicio core subyacente, proveyendo de forma estable los flujos de información y los datos de materiales indexados en pesos chilenos y Unidades de Fomento (UF) que permitieron la persistencia y la exportación documental en PDF validadas en las capas de Backend y Frontend de este sprint.

## 4 ESTRATEGIA DE EJECUCIÓN DE PRUEBAS

La estrategia de QA seguida en el proyecto ConstructoCompare Pro se basa en los siguientes principios:

### 4.1 Tipos de Prueba Utilizados

- Pruebas de Humo (Smoke Tests): verificación rápida de la funcionalidad crítica antes de pasar a pruebas más extensas.
- Pruebas de Estrés: evaluación del comportamiento bajo carga elevada (conurrencia de usuarios y volumen de datos).
- Pruebas de Volumen: inyección masiva de datos para detectar degradaciones de rendimiento o congelamientos.
- Pruebas de Regresión: validación de que funcionalidades previas se mantienen estables tras nuevas entregas.
- Pruebas de Fuego/Spike: picos abruptos de concurrencia para evaluar la estabilidad del hosting.

### 4.2 Herramientas

- Pytest: framework principal de pruebas unitarias e integración para backend (Python/FastAPI).
- Playwright: automatización de pruebas E2E en frontend (Next.js) y scraping (Sodimac, Easy, Imperial).
- HTTPX + asyncio: cliente HTTP asíncrono para pruebas de carga sobre endpoints de la API.
- Locust: pruebas de carga masiva con usuarios simulados concurrentes.
- psutil: monitoreo de recursos de hardware (CPU y RAM) durante pruebas de estrés.
- MagicMock / AsyncMock: simulación de dependencias para pruebas de backend aisladas.

### 4.3 Criterios de Aceptación Generales

- Backend: 0 errores internos de servidor (HTTP 500) bajo carga concurrente.
- Frontend: tiempo de respuesta < 5 s en operaciones normales; < 10 s en pruebas de estrés de login.
- Scraping: campos mínimos obligatorios (nombre, SKU, precio) presentes en todos los registros extraídos.
- Datos (Capa Silver): 100 % de precios en formato numérico puro; 0 strings con espacios dobles.

## 5 ANEXOS

Toda la evidencia de pruebas ejecutadas durante los Sprints 1 a 5 se encuentra consolidada en el siguiente documento:

**ANEXO QA Pruebas\_de\_Criterios\_de\_Aceptación.docx** Autor: Genara Alarcón (QA Engineer / DevOps) Período: 30/03/2026 – 31/05/2026

Dicho documento contiene:

- Anexo A: Reportes de consola de Pytest por sprint
- Anexo B: Informes de Locust (RPS, latencias y tasas de fallo)
- Anexo C: Capturas de pantalla de pruebas E2E con Playwright
- Anexo D: Métricas de hardware (CPU/RAM) de pruebas de estrés de scraping

## 6 GLOSARIO

| Término            | Definición                                                                                                          |
|--------------------|---------------------------------------------------------------------------------------------------------------------|
| AC                 | Criterio de Aceptación (Acceptance Criterion). Condición que un sistema debe satisfacer para ser aceptado.          |
| API                | Application Programming Interface. Interfaz de programación de aplicaciones.                                        |
| Asyncio            | Biblioteca de Python para programación asíncrona mediante corutinas.                                                |
| Backend            | Capa de servidor de la aplicación. En este proyecto: FastAPI (Python).                                              |
| Bronze/Silver/Gold | Capas del pipeline Medallion: Bronze (datos brutos), Silver (datos normalizados), Gold (datos listos para negocio). |
| E2E                | End-to-End. Prueba que recorre el sistema completo desde la interfaz hasta la base de datos.                        |
| FastAPI            | Framework de Python para construcción de APIs REST de alto rendimiento.                                             |
| Frontend           | Capa de interfaz de usuario. En este proyecto: Next.js (React).                                                     |
| HU                 | Historia de Usuario. Requisito funcional expresado desde la perspectiva del usuario.                                |
| HTTPX              | Cliente HTTP asíncrono para Python utilizado en pruebas de carga de la API.                                         |
| JWT                | JSON Web Token. Estándar de autenticación basado en tokens firmados.                                                |
| Lazy Loading       | Carga perezosa. Técnica de carga de datos bajo demanda al hacer scroll.                                             |
| Locust             | Herramienta de pruebas de carga que simula múltiples usuarios concurrentes.                                         |
| Medallion          | Arquitectura de datos en capas (Bronze, Silver, Gold) para procesamiento incremental.                               |
| Matching Semántico | Algoritmo que agrupa productos equivalentes de distintas tiendas bajo una identidad canónica.                       |
| Playwright         | Framework de automatización de navegadores para pruebas E2E.                                                        |
| psutil             | Librería Python para monitoreo de recursos del sistema (CPU, RAM, disco).                                           |
| pytest             | Framework de pruebas unitarias y de integración para Python.                                                        |
| QA                 | Quality Assurance. Aseguramiento de la calidad del software.                                                        |
| Scraping           | Extracción automatizada de datos desde sitios web (web scraping).                                                   |
| SKU                | Stock Keeping Unit. Identificador único de producto en el catálogo de una tienda.                                   |

|            |                                                                                                     |
|------------|-----------------------------------------------------------------------------------------------------|
| Smoke Test | Prueba de humo. Verificación rápida de que las funciones críticas del sistema operan correctamente. |
| Sprint     | Iteración de desarrollo en metodologías ágiles, típicamente de 1 a 4 semanas.                       |
| UF         | Unidad de Fomento. Unidad de cuenta chilena indexada al IPC.                                        |